Advanced Git: collaborating with others

Laura Prieto Saavedra
Scientist and Research Software Engineer
This work is licensed under a CC BY 4.0 license.

April 09, 2026, Software and Data Seminar

Context

Why is this relevant for you?

- If you already know all the basics of Git, how to share it with others?
- How to keep a copy of your project on a Git-hosting system?
- What workflow should I use for collaboration?

Goal: to learn advanced collaboration tools with Git and define a workflow.

How? This session will be **interactive**. I will present the concepts and show you how to use them. You will also be able to try them out in your computer if you have a complete setup.

Note: the material is based on previous seminars, CodeRefinery lessons and the Git Pro book available online `https://git-scm.com/book/en/v2`.

Prerequisites: what do you need?

A laptop with internet access and a (`bash`) terminal.

Working Git installation: verify by typing in your terminal `git --version`.

Internal WIAS GitLab:

- Credentials: username, password and second-factor authentication code.
- SSH key configured in your computer.

IDE (Optional): we will be looking at VSCode.

Materials: download sample directory from the seminar chat (same as introduction to Git).

During the presentation:

- Warnings will be shown in red.
- Best practices will be shown in orange.
- Tasks for you will be shown in blue.

Content

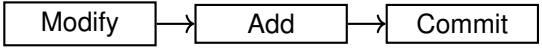
1. Recap
2. GitLab
3. Interaction commands
4. Collaboration workflow
5. What comes next?

Content

1. Recap
2. GitLab
3. Interaction commands
4. Collaboration workflow
5. What comes next?

Recap

In the previous seminar, we learned all the basics of Git in the command line and working on a local repository. The main workflow is:



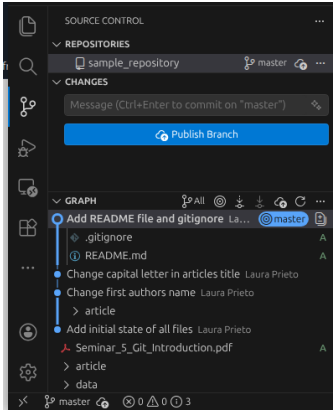
The most important commands ones are:

- `git init` to initialize your repository.
- `git status` to check the repository status and the branch.
- `git log` to check the commit history.
- `git add file` to add a modified file.
- `git commit -m "message"` to create a snapshot of the project.
- `git switch -c name_of_new_branch` to create a new branch.

Why is this important if I can do everything using a nice interface given by another program or IDE? The basic concepts and workflow is the same.

IDE example: VSCode

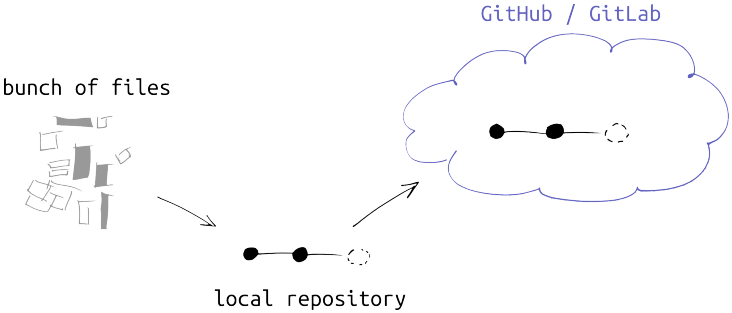
- Built-in Git support.
- Integrated terminal (I can always go back to CLI if needed).
- Useful to perform all the same tasks and with similar (almost exactly the same) vocabulary.
- Cross-platform: works in Windows, macOS and Linux.



Other options: GitKraken, Sourcetree, Github Desktop.

Today's goal

How to share our work with others?



This is one of the main strengths of Git. **Let us get familiar with the Git-hosting system supported by WIAS: GitLab.**

Additional advantages: have backup of your work or make it findable.

Content

1. Recap
2. GitLab
3. Interaction commands
4. Collaboration workflow
5. What comes next?

GitLab

It is one web-based platform for managing Git repositories.

Main characteristics:

- store repositories remotely
- collaborate
- review changes
- automate builds and deployments (code)
- manage projects

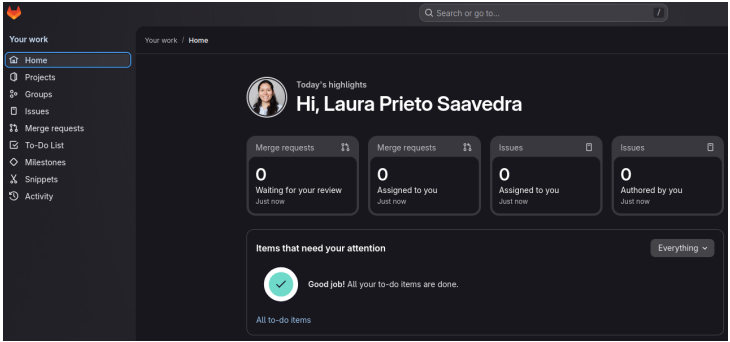
GitLab works in two ways:

- On the cloud (just like GitHub).
- Self-hosted (like at our institute).

Self-hosted allows for full data control, private infrastructure and compliance requirements.

Log into WIAS GitLab

Go to the WIAS internal GitLab URL, use your WIAS credentials and your second-factor authentication app.

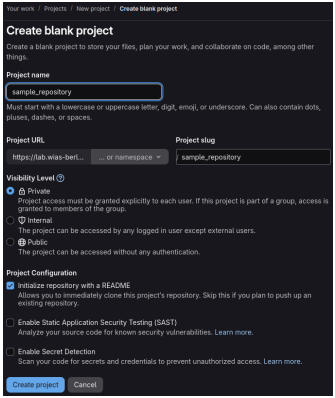


Documentation is on the left bottom corner.

Creating a project

We create a remote repository as follows:

1. Click **Projects** on the left-hand side panel.
2. Create project by clicking on **New Project**.
3. Choose a blank project.
4. Type a project name, set visibility level and initialize repository with a `README` file (optional).



Best practices: ideal project names include only lowercase letters and no spaces.

Warning: set the visibility level according to your project needs.

Task: create a new project on your own GitLab account.

What to do once a project is created?

The instructions are listed if the repository is empty:

Option 1: create a new repository and clone
You take the repository you have just created in the server and clone (copy) to your local machine.

Option 2: push an existing repository
You already have a directory that was initialize locally and want to simply push (copy) it to the server.

In general, in order to interact with GitLab from your local machine, you will need an SSH key or a token.
Best practices: it is always more secure to use `ssh`. Sometimes convenient to have a token, BUT not all tasks might be available.

Option 1: create a new repository and clone

You created a remote repository and need a local version to start working locally. For this you use the following command:

Command 1: `git clone path_to_remote_repository`

- The path can be found by clicking on the the blue “Code” button under clone with SSH.

Warning: the repository will be cloned in the directory you currently are when executing the command in the terminal.

If your project is not public, you will need to give explicit access to members of your group. In the left-hand side panel, click on Manage → Members → Invite members.

Task: clone the repository with the materials of this seminar.

Option 2: push an existing repository

You already have a Git directory and want to push it to the new remote repository. For this we use the following commands:

Command 2: `git remote add origin path_to_remote_repository`

- You can check what this command did by typing `git config --list`.
- We basically added a remote url to the repository and called it “origin".

Command 3: `git push --set-upstream origin --all`

- By using the flag --all, we are adding all the branches to the remote repo.

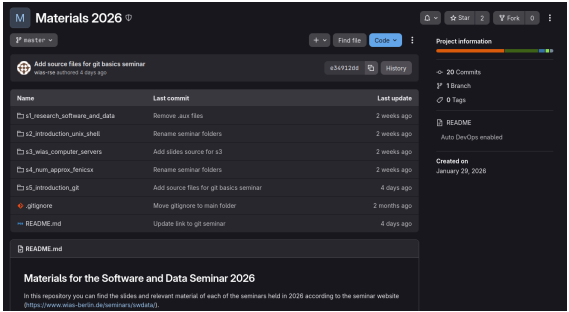
Warning: this will only work if you have set your Git identity (username and email) and SSH key correctly.

Task: push the sample repository to the newly created repository.

A Git repository with a nice interface

What information do we have?

- List of all files with the last commit and update.
- List of branches under the name of the repository.
- History of your repository with metadata and commit ID.
- Markdown version of your `README` file.
- A summary of commits, branches and tags on the right hand side.
- ...



How do I interact with it? How do I send my local changes? How do I update my local repository once someone has made changes?

Content

1. Recap
2. GitLab
3. Interaction commands
4. Collaboration workflow
5. What comes next?

Push update to remote repositories

Scenario: you continue working on your local repository. You want to update the remote repository. How to do it?

To push a specific branch to the remote origin:

Command 4: `git push origin specific_branch`

- Without specifying the branches, one can push the current branch only by typing `git push`. If you are pushing a branch that you have never pushed before you need to add a flag: `git push -u origin specific_branch`, where `-u` is the same as `--set-upstream`.

Task: create a new branch, perform some changes and push it to the remote repository. Then merge this branch into your master branch and push the changes.

Pulling or fetching updates from remote repositories

Scenario: you shared your repository and someone made changes. You want to update your local repository. How to do it?
To fetch the changes but do not change any of your local branches:

Command 5: `git fetch origin specific_branch`

- Updates remote information and allows you to inspect changes in your computer.

To fetch the changes and at the same time rebase your current branch:

Command 6: `git pull --rebase`

To fetch the changes and at the same time merge them into your current branch:

Command 7: `git pull`

Task: add something to the `.gitignore` in Gitlab. Fetch the changes locally and pull.

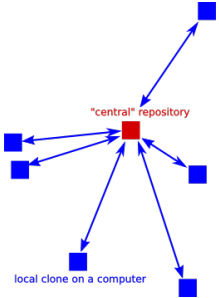
Content

1. Recap
2. GitLab
3. Interaction commands
4. Collaboration workflow
5. What comes next?

Collaboration workflow

Once you have both a local and a remote repository. How do you actually collaborate on your project?

Centralized workflow: there is a central remote repository, every team member can **clone** it. Every member works locally, exchange commits through the remote (using Merge Requests) and synchronize regularly.



Taken from CodeRefinery.

Workflow useful for small projects with specific collaborators working in a team.

First step: add team members and define their role

Persons involved: owner of the repository.

To add members on GitLab use the following steps:

1. Click **Manage** on the left-hand side panel.
2. Access the list of team members by clicking **Members**.
3. Click on the blue **Invite members** button and write the name of your collaborator.

Roles:

1. Maintainer (project owner): person in charge of managing branches, merge requests (changes) and adding project members.
2. Developer (collaborator): person that is allowed to add changes via branches and creation of merge requests.

Task: give me a few usernames and I will add you as developers to my repository. Check the project on your GitLab account, you should also get an email with the invitation.

Second step: modify something and create a merge request

Persons involved: the collaborators or developers.

What is a merge request? a change proposal.

Steps to create a merge request:

1. Create a new branch with a meaningful name.
2. Modify the files you wish to change.
3. Commit the changes.
4. Publish (push) the new branch.
5. Submit a merge request → click the “Create merge request" blue sign that appears in GitLab.

Task: follow the steps to create your new branch. You can do everything on GitLab, or you can do 1-4 in your computer using the commands previously learned.

Third step: review changes and merge

Persons involved: the maintainer or the owner.

Steps to review changes:

1. Discuss your changes.
2. Wait for update if changes are needed.
3. Repeat the previous two steps until everyone is happy.
4. Merge the changes into the master branch.

Reviewing and merging requests is a great way of learning to use Git and becoming and expert in conflict resolution.

Best practices: when working in larger teams, allow more team members to review and merge changes.

Task: let us review and merge requests together.

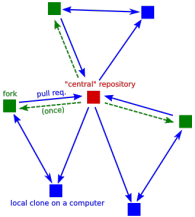
Some FAQs

- **Once a merge request (MR) is open how can I commit to it?**
  Just continue to contribute in the corresponding branch as usual. The changes will be reflected in the MR.
- **Is there a limit for branches?**
  No, but if working in a larger team, `forking` might be more appropriate (coming soon).
- **Why cannot we all push to the master branch?**
  You can change the settings of the project but this is NOT recommended. It will lead to all members of the team solving conflicts all the time. This workflow also ensures that the changes can be reviewed by other person.
- **What am I supposed to review?** Typos, whether you understand and agree with the changes, whether you have other suggestions or things to correct before merging the changes to the master branch.

Other workflow

Forking workflow: there is a central remote repository with the main members of the project team. If other persons want to contribute, they have to **fork** the remote repository and propose changes through there.

To fork a repository, simply search for the “Fork" button next to the title and the star. You will get immediately a copy of the whole repository to a new remote in your own GitLab namespace.



Taken from CodeRefinery.

Workflow useful for large open-source projects, public repositories and large communities.

Content

1. Recap
2. GitLab
3. Interaction commands
4. Collaboration workflow
5. What comes next?

Final remarks

To conclude:

- Git coupled with GitLab is a very powerful tool for your individual or team projects.
- While there are many commands, the only way of learning them is to use them on your projects. Learn the basics and go from there according to your needs.
- Reach out if you want to learn something more specific or if you want a Git seminar focused on code (automation and deployment).

Thank you for your attention!

Software and Data Seminar

References

Pro Git book.
https://git-scm.com/book/en/v2.
[Accessed 01-04-2026].

Bast, R., Wikfeldt, K. T., Darst, R., Razick, S., Iusan, D., Lindi, B., Andrei, C.-O., Suvilehto, J., and Fasshauer, E. (2025a).
Introduction to version control with git - CodeRefinery lesson.

Bast, R., Wikfeldt, K. T., Hellsvik, J., Darst, R., Razick, S., Pushpadas, D., Lindi, B., Pilstål, R., Alim Rasel, A., Suvilehto, J., and Fouilloux, A. (2025b).
Collaborative distributed version control - CodeRefinery lesson.